

AI-NATIVE ENGINEERING

Architecture Guide for a Model-Driven Customer Onboarding System

A beginner-friendly explanation of Frontend, Backend, Runtime Architecture, Generative UI, AI Runtime, and the Engineering Harness

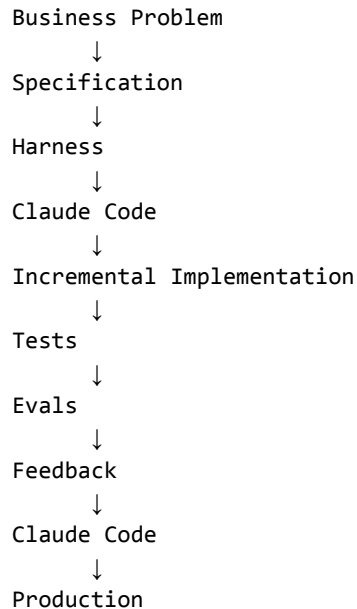
Purpose of this document

This document explains the architecture discussion from first principles. It shows why a modern AI-native web application should still use familiar industry concepts such as frontend and backend, while also introducing logical runtime views such as UI Runtime, Domain Runtime, and AI Runtime. The goal is to build a Customer Onboarding web application while learning AI-native engineering and harness engineering with Claude Code.

1. The Big Picture

Our application is a web application for customer onboarding. A customer uses a browser to start an application, provide information, upload documents, and eventually receive a decision or a request for human review. The system also uses AI models for tasks such as document understanding, validation, risk analysis, and selected Generative UI experiences.

The overall engineering lifecycle we are studying is:



The key idea is that Claude Code is not the customer-facing application. Claude Code is the AI software-engineering agent that helps us build and improve the application. The runtime AI models inside the application are separate components.

2. First Principle: What Are We Actually Building?

We are building an enterprise Customer Onboarding System. It is intentionally a realistic example because it combines ordinary software engineering with probabilistic AI behaviour.

- Customer registration and application creation
- Secure document upload
- Document information extraction
- Cross-document validation and discrepancy detection
- AI-assisted risk assessment
- Human review for uncertain or sensitive cases
- Approval or rejection through controlled business workflows
- Auditability and traceability

3. Why the Architecture Question Matters

A beginner may see an application as simply a frontend and a backend. That is a valid starting point, but an AI-native system has another important dimension: some components are probabilistic. The model may

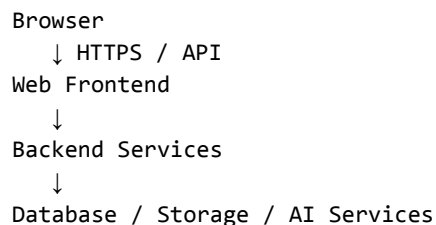
interpret the same type of input in different ways, may be uncertain, and may require evaluation rather than only conventional unit testing.

Important distinction

Frontend/backend and UI/domain/AI are not competing standards. They describe different architectural views. Frontend/backend is a useful system or implementation view. UI/Domain/AI is a logical responsibility view. A runtime view explains what responsibilities execute while the application is running.

4. Is Frontend + Backend Still Industry Standard?

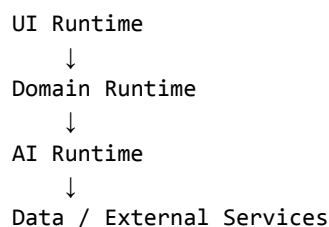
Yes. For an enterprise web application, it is completely normal and useful to describe the system as a web frontend communicating with backend services through APIs. We should not throw this model away just because AI is involved.



For our project, React + TypeScript will be a practical choice for the web frontend, while Python + FastAPI can provide the backend API and domain/application services. These technology choices are implementation details, not the entire architecture.

5. Then Why Introduce Runtime Architecture?

Runtime architecture gives us a more useful way to explain what the system actually does. Instead of asking only 'Which technology is this?', we ask 'What responsibility is executing here?'

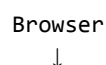


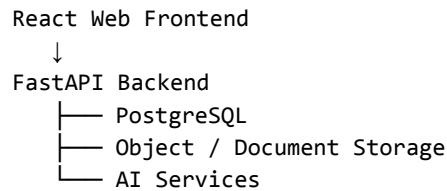
This does not mean that the industry has replaced frontend/backend terminology with these three terms. Rather, we use the runtime view to make the AI-native behaviour explicit.

6. Three Architectural Views We Will Use

6.1 System / Deployment View

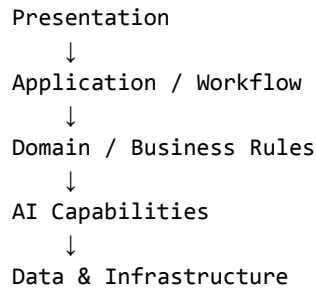
This answers: 'What applications and services are deployed and communicating with each other?'





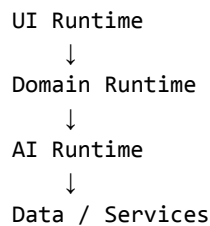
6.2 Logical Architecture View

This answers: 'What responsibilities exist inside the system?'



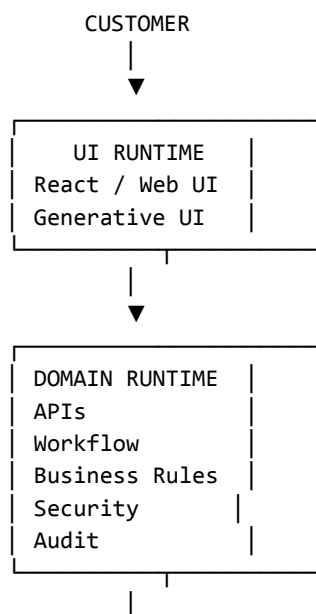
6.3 AI-Native Runtime View

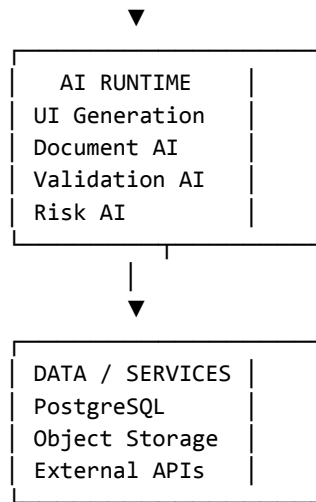
This answers: 'How do the major responsibilities execute together at runtime?'



7. Our Recommended Architecture

For the Customer Onboarding project, we should use all three views rather than choosing only one. This keeps the architecture industry-friendly while making the AI-native design clear.





8. What Is the UI Runtime?

The UI Runtime is the part of the running application responsible for what the user sees and interacts with in the browser. In our project, React and TypeScript will implement most of this responsibility.

It can contain both deterministic and generative UI.

Deterministic UI

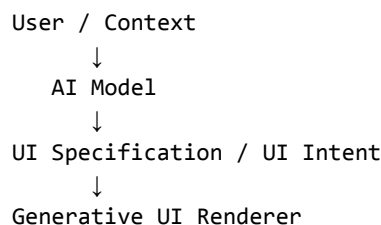
- Login and authentication screens
- Navigation
- Document upload controls
- Required disclosures
- Approval controls
- Security-sensitive actions

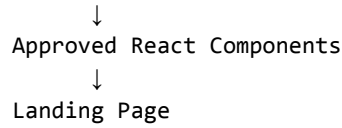
Generative UI

- Dynamic landing-page composition
- Personalized onboarding guidance
- Contextual help
- Dynamic explanations
- Context-aware review panels

9. Generative UI in Our Landing Page

We agreed that the landing page is an excellent first demonstration of a model-driven application. Instead of hard-coding every part of the page, an AI model can generate a controlled UI specification. The browser application then renders that specification using approved components.





The important safety principle is that the model should not be allowed to generate arbitrary executable code. The model selects from an approved component vocabulary, while the application controls how those components are rendered.

Approved Components

Hero
Heading
Text
FeatureCard
FeatureGrid
CTA
Form
DocumentUpload
Alert
ProgressIndicator
ReviewPanel

10. Example of a UI Contract

The AI model could return a structured representation such as the following. The exact schema will be designed later.

```
{
  "page": "customer_onboarding",
  "components": [
    {"type": "hero", "title": "Start Your Customer Onboarding"},
    {"type": "feature_grid", "items": [
      "Secure document verification",
      "Fast application processing",
      "Human-assisted review"
    ]},
    {"type": "cta", "label": "Start Application"}
  ]
}
```

The renderer validates the structure and maps each approved component type to a known React component. This is safer than asking the model to write arbitrary HTML or JavaScript.

11. What Is the Domain Runtime?

The Domain Runtime is where the application's core business process is controlled. It is responsible for things that should be predictable, enforceable, auditable, and governed by business rules.

- Customer onboarding workflow and state
- Business rules
- API orchestration
- Authentication and authorization integration
- Permissions

- Audit records
- Human-review routing
- Validation of AI outputs before important actions

Why this boundary matters

The AI should normally recommend or interpret; the Domain Runtime decides whether an action is allowed. For example, an AI model can identify a high-risk indicator, but the domain workflow can require human review rather than allowing the model to directly reject a customer.

12. What Is the AI Runtime?

The AI Runtime contains the runtime components that invoke and manage AI models. This is where probabilistic behaviour is treated as a first-class engineering concern.

- Model gateway / model integration
- Prompt and context management
- Structured outputs
- UI generation
- Document understanding
- Validation and discrepancy analysis
- Risk analysis
- Tool calling and agent orchestration

13. Deterministic vs Probabilistic Boundaries

One of the most important principles in our architecture is to decide deliberately which responsibilities should remain deterministic and which can use probabilistic AI.

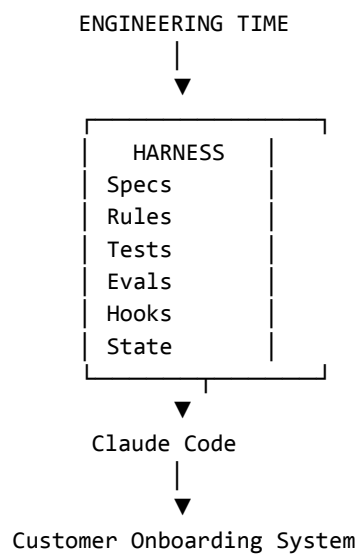
DETERMINISTIC
 Authentication
 Authorization
 Workflow state
 Database transactions
 Audit logging
 Required fields
 Security controls
 Business rules

↓ controlled boundary ↓

PROBABILISTIC
 Document interpretation
 Semantic comparison
 Classification
 Risk reasoning
 Natural-language understanding
 Dynamic UI composition

14. Where Does Claude Code Fit?

Claude Code belongs primarily to the engineering lifecycle, not to the customer-facing runtime. It is the AI coding agent that helps us create, inspect, test, modify, and improve the application.



15. What Is the Harness?

The harness is the engineered environment around the coding agent. It gives Claude Code the project context, constraints, tools, feedback mechanisms, and verification mechanisms needed to work effectively over multiple iterations.

- CLAUDE.md and project context
- Specifications and model contracts
- Rules and permissions
- Hooks and controlled actions
- Skills or specialized agent instructions
- Tests
- Evaluation datasets and rubrics
- Git history and project state
- Feedback from failures and production

A crucial distinction is that CLAUDE.md is guidance and project context; it should not be treated as a hard security boundary. Permissions, hooks, and application-level controls provide stronger enforcement.

16. Frontend/Backend vs Runtime Architecture

View	Main Question	Example	Our Use
System / Deployment	What is deployed?	Frontend, backend, database, AI service	Yes
Logical	What responsibilities exist?	Presentation, domain, AI, data	Yes

Runtime	What executes together?	UI Runtime, Domain Runtime, AI Runtime	Yes
Technology	What implements it?	React, FastAPI, Python, PostgreSQL	Yes

17. The Final Architecture Vocabulary We Will Use

Recommended terminology

Primary system view: Web Frontend + Backend Services + AI Services.

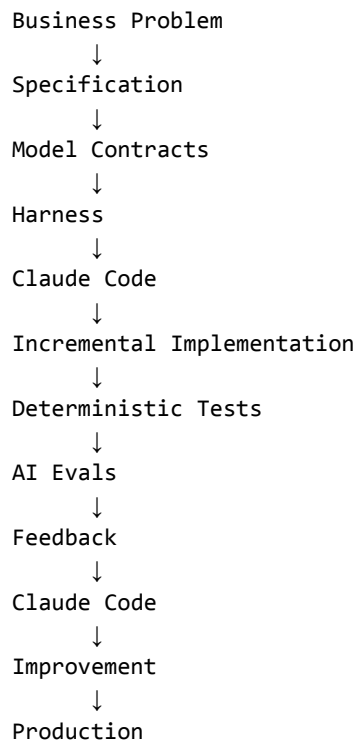
Logical view: Presentation + Application/Domain + AI + Data.

AI-native runtime view: UI Runtime + Domain Runtime + AI Runtime.

Engineering layer: Harness + Claude Code + Tests + Evals + Feedback.

18. How This Connects to AI-Native Engineering

The architecture now gives us a clean place to introduce AI-native engineering. We are not merely adding an LLM API to a conventional application. We are designing the system so that model behaviour, uncertainty, evaluation, and feedback are part of the engineering lifecycle.



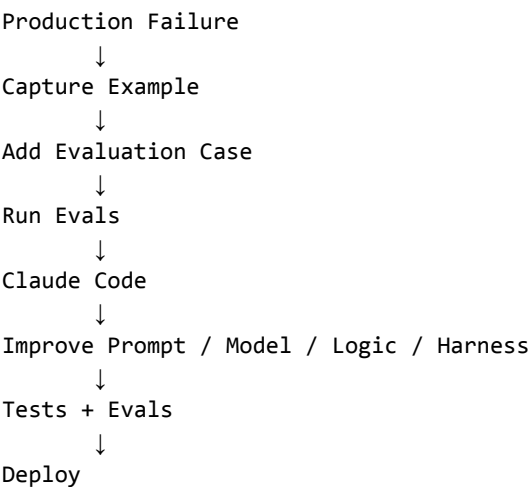
19. Why Tests and Evals Are Both Needed

Traditional tests are excellent for deterministic software behaviour. AI evaluations are needed to measure probabilistic behaviour. A customer onboarding system needs both.

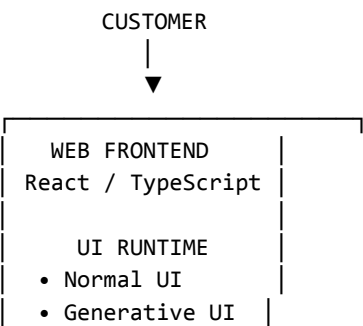
Tests	Evals	Example
Deterministic correctness	AI behaviour quality	API returns correct HTTP status vs. model extracts the correct document number
Regression tests	Golden/evaluation cases	Workflow still works vs. model handles ambiguous documents acceptably
Code-level failures	Model/prompt/context failures	Database error vs. unsupported AI claim

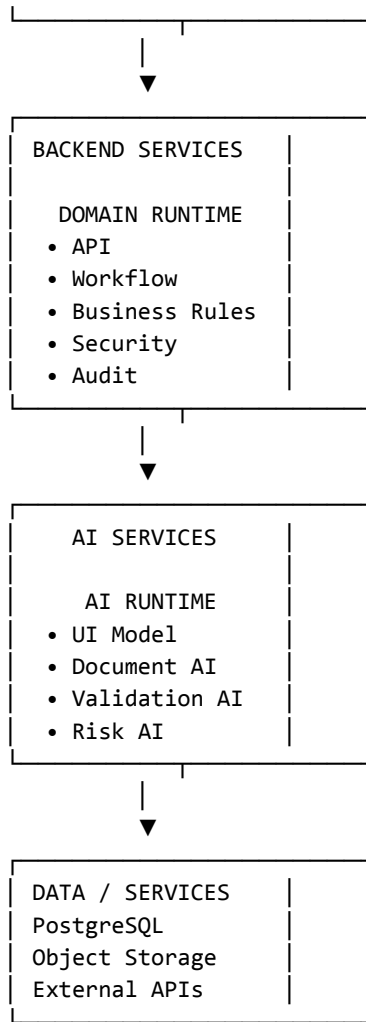
20. Feedback as an Engineering Asset

When a production case exposes an AI failure, we should not merely fix the immediate symptom. The case can become a new evaluation scenario. This gradually strengthens the evaluation corpus.

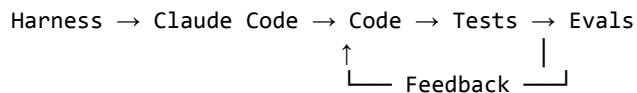


21. The Complete Customer Onboarding Architecture





ENGINEERING SIDE



22. Beginner Mental Model

If all of this feels like too many architectural terms, remember these four simple questions:

1. What does the customer see? → UI / Frontend
2. What controls the business process? → Domain / Backend
3. What responsibilities require probabilistic intelligence? → AI Runtime
4. What helps Claude Code build and improve the system safely? → Harness

23. What We Should Build Next

We should now stop changing the conceptual architecture and start turning it into an executable project. The next practical step is to create the repository from scratch and define the first version of the specification and harness.

```
customer-onboarding-ai/
├── CLAUDE.md
├── README.md
├── docs/
│   ├── architecture.md
│   └── business-problem.md
├── specs/
│   ├── spec.md
│   └── model-contracts.md
├── .claude/
│   ├── rules/
│   ├── hooks/
│   └── agents/
├── frontend/
├── backend/
├── tests/
├── evals/
│   ├── datasets/
│   ├── scenarios/
│   └── rubric.md
```

The first model-driven feature should be the landing page with controlled Generative UI. That gives us a small, visible example through which we can demonstrate the complete AI-native cycle before expanding into document processing, validation, and risk assessment.

24. Final Takeaway

The architectural decision

We should NOT replace frontend/backend architecture with runtime terminology. Instead, we use frontend/backend as the practical system/deployment view, logical layers to describe responsibilities, and UI Runtime / Domain Runtime / AI Runtime as an AI-native runtime view. This is both understandable to beginners and credible for enterprise architecture discussions.

**Business Problem → Specification → Harness → Claude Code → Implementation → Tests → Evals →
Feedback → Claude Code → Production**